

Revisão Geral para a Maratona SBC

Lucas A. Moraes

¹FATEC Itu

1. Introdução aos Grafos

1.1. Ideia de Grafos

Os grafos são estruturas de dados que tem como ideia principal criar relacionamentos em um mesmo ambiente ou ambientes diferentes (grafos bipartidos). Através dessa estrutura de dados, podemos representar computacionalmente e matematicamente relações como por exemplo: interações em redes sociais, caminhos entre cidades, fluxo de encanamentos etc.

Os grafos possuem dois conceitos básicos:

- **Vértices:** Os vértices são a estrutura principal apresentada, ele é quem determina qual é o "universo" que estamos estudando. Exemplos de vértices: cidades, pessoas para redes sociais, pontos de articulação em encanamento etc.
- **Arestas:** As arestas representam as ligações (relacionamentos) entre os vértices, elas podem ser **ponderadas** ou não (isto é, terem valores ou não). As arestas podem representar diversos tipos de relacionamentos, por exemplo em um grafo composto por vértices representando cidades, as arestas entre dois vértices podem representar a existência de um caminho da cidade U para V tendo um valor de W.

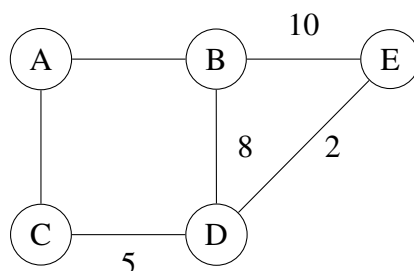


Figura 1. Exemplo de um grafo simples ilustrando vértices e arestas (algumas ponderadas).

1.2. Definições de Grafos

Abaixo estão apresentados algumas definições de grafos, para entendimento de outros assuntos posteriores.

- **Grafo Esparso:** Um grafo onde o número de arestas (E) é muito menor que o quadrado do número de vértices (V^2). Geralmente, dizemos que $E \approx V$.
- **Grafo denso:** Ao contrário do esparso, aqui o número de arestas se aproxima de V^2 .
- **Ciclo:** Um caminho que começa e termina no mesmo vértice, sem repetir arestas.

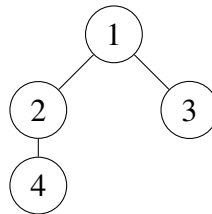
- **Grau de um vértice:** O número de arestas incidentes a um vértice. Em grafos direcionados, separamos em grau de entrada (*in-degree*) e grau de saída (*out-degree*).
- **Caminho:** Uma sequência de vértices onde cada par consecutivo está conectado por uma aresta.

2. Grafos Computacionalmente

A seguir iremos abordar alguns algoritmos de grafos, para facilitar o entendimento em geral, será optado pelo uso de pseudo-linguagem nas explicações, seguidos dos códigos em C++ e Python.

2.1. Definição Computacional de Grafos

Os grafos podem ser definidos computacionalmente de duas formas, veremos as representações usando o grafo abaixo.



- **Lista de Adjacência:** A lista de adjacência é a maneira mais comum, ela funciona de uma maneira bem simples em cada posição i da lista de adjacência haverá uma lista de vértices associados ao vértice i . Vejamos um exemplo:

i	Adjacências
1	[2, 3]
2	[1, 4]
3	[1]
4	[2]

- **Matriz de Adjacência:** é um pouco menos utilizada, mas basicamente as matrizes de adjacência tem uma ideia semelhante as listas de adjacência. Dado um grafo com N vértices, você criará uma tabela (matriz) $N \times N$ em que cada linha representa um vértice i , e em cada coluna j , se o valor for diferente de 0, significa que há uma aresta de $i \rightarrow j$

Matriz	1	2	3	4
1	0	1	1	0
2	1	0	0	1
3	1	0	0	0
4	0	1	0	0

Podemos trocar e usar valores diferentes de 1 e representar grafos ponderados,

3. Algoritmos de Grafos

A seguir iremos abordar alguns algoritmos de grafos, para facilitar o entendimento em geral, será optado pelo uso de pseudo-linguagem nas explicações, seguidos dos códigos em C++ e Python.

3.1. Algoritmos de Busca

3.1.1. DFS - Depth First Search (Busca em Profundidade)

A busca em profundidade utiliza de recursão para visitar os nós de um grafo, sempre indo para o nó mais profundo no grafo e quando não tiver mais nós vizinhos para serem visitados, faz o processo de **backtracking** (retornar a chamada recursiva anterior). Sua complexidade é: $\mathcal{O}(V + E)$

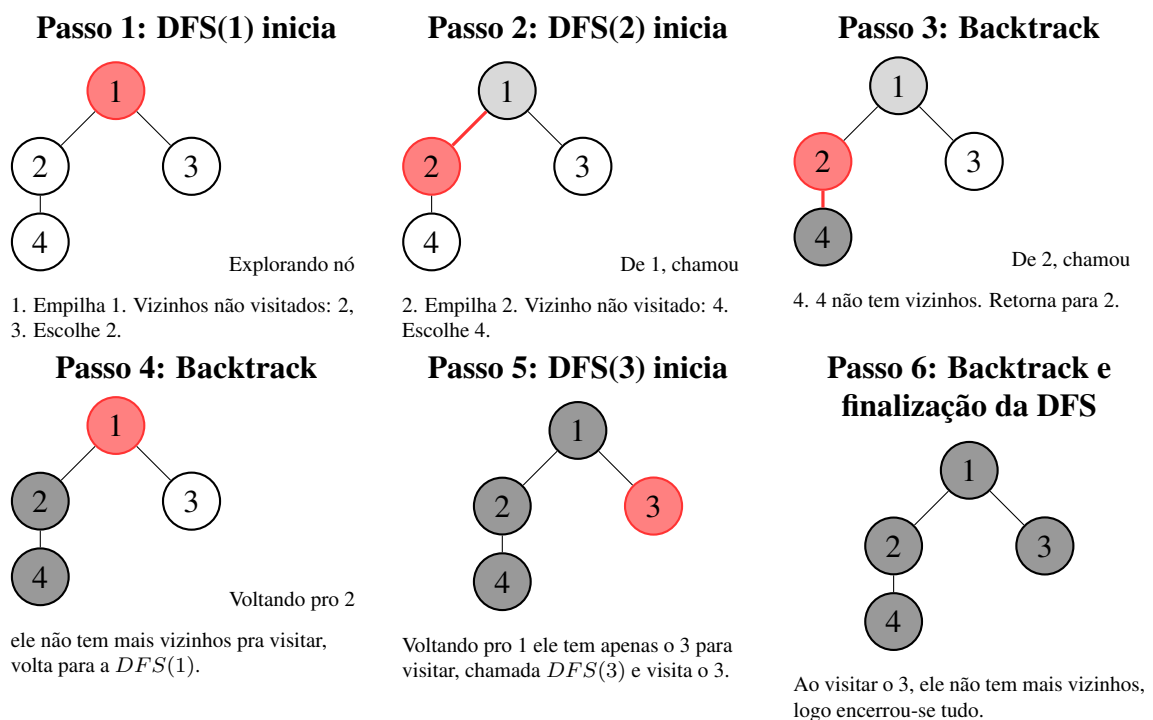
Pseudo-Código:

Algoritmo 1: Busca em Profundidade (DFS)

```
1 Função DFS ( $v, adj, visitado$ ) :  
2    $visitado[v] \leftarrow \text{true}$ ;  
3   processar  $v$ ;  
4   para cada  $u \in adj[v]$  faça  
5     se  $\neg visitado[u]$  então  
6       DFS ( $u, adj, visitado$ ) ;  
7   fim  
8 fim
```

O funcionamento da DFS é bem intuitivo, inicialmente passamos um vértice qualquer (geralmente os grafos são enraizados, então passamos a raiz que por convenção é "labelada" em 0), o vetor de adjacências (isto é o grafo) e o vetor de visitados (um vetor de booleanos que marca os vértices que já foram vistos, isto é importante para evitar loops infinitos).

Veja a DFS funcionando de maneira mais visual, iniciando rodando $DFS(1)$:



Veja que nessa ordem de visita  o temos: [1, 2, 4, 3].

Implementa  o:

Veamos a implementa  o da DFS em Python e em C++.

Python:

```
1
2 def dfs(u, adj, visited):
3     visited[u] = True
4     print(u) # Processa o vertice atual
5
6     for v in adj[u]:
7         if not visited[v]:
8             dfs(v, adj, visited)
9
```

C++:

```
1
2 void dfs(int u, const vector<vector<int>>& adj, vector<bool>& visited) {
3     visited[u] = true;
4     // Processa o vertice atual aqui
5     for (int v : adj[u]) {
6         if (!visited[v]) {
7             dfs(v, adj, visited);
8         }
9     }
10 }
```

3.1.2. BFS - Breadth First Search (Busca em largura)

A busca em largura explora o grafo nível por nível, visitando primeiro todos os vizinhos diretos de um nó antes de avançar para os vizinhos dos vizinhos. Para isso, utiliza uma estrutura de dados de fila (*Queue* - FIFO). Sua complexidade é: $\mathcal{O}(V + E)$

Pseudo-Código:

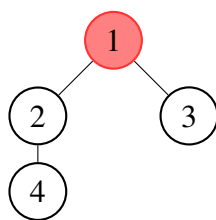
Algoritmo 2: Busca em Largura (BFS)

```
1 Função BFS (inicio, adj) :  
2   fila  $\leftarrow$  nova Fila();  
3   visitado  $\leftarrow$  novo vetor de booleanos;  
4   fila.push(inicio);  
5   visitado[inicio]  $\leftarrow$  true;  
6   while enquanto fila não vazia do  
7     v  $\leftarrow$  fila.front(); fila.pop();  
8     processar v;  
9     para cada u  $\in$  adj[v] faça  
10      se  $\neg$ visitado[u] então  
11        visitado[u]  $\leftarrow$  true;  
12        fila.push(u);  
13      fim  
14    fim  
15  end
```

O funcionamento da BFS garante que encontramos o caminho mais curto em grafos não ponderados, pois ela expande as frentes de busca de forma uniforme.

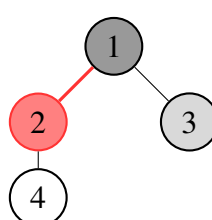
Veja a BFS funcionando de maneira mais visual, iniciando em $BFS(1)$:

Passo 1: Início



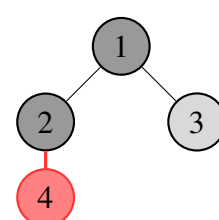
Insere 1 na fila. O nó 1 é o atual. Fila: [1].

Passo 2: Expandindo 1



Remove 1. Visita e insere os vizinhos 2 e 3 na fila. Fila: [2, 3].

Passo 3: Expandindo 2



Remove 2. Visita e insere o vizinho 4. Fila: [3, 4].

Veja que a ordem de visitação por níveis da BFS resulta em: [1, 2, 3, 4].

Implementação: Vejamos a implementação da BFS em Python e em C++.

Python:

```
1  from collections import deque
2
3  def bfs(inicio, adj):
4      visitado = [False] * len(adj)
5      fila = deque([inicio])
6      visitado[inicio] = True
7
8      while fila:
9          v = fila.popleft()
10         print(v) # Processa o vertice atual
11         for u in adj[v]:
12             if not visitado[u]:
13                 visitado[u] = True
14                 fila.append(u)
```

C++:

```
1  void bfs(int inicio, const vector<vector<int>>& adj) {
2      vector<bool> visited(adj.size(), false);
3      queue<int> q;
4
5      q.push(inicio);
6      visited[inicio] = true;
7
8      while (!q.empty()) {
9          int v = q.front();
10         q.pop();
11         // Processa o vertice atual aqui
12
13         for (int u : adj[v]) {
14             if (!visited[u]) {
15                 visited[u] = true;
16                 q.push(u);
17             }
18         }
19     }
20 }
```

3.2. Algoritmos de Caminho Mínimo de Fonte Única

3.2.1. Dijkstra - Caminho Mínimo com Pesos Não Negativos

O algoritmo de Dijkstra encontra o caminho mais curto a partir de um nó origem para todos os outros nós em um grafo com arestas de pesos não negativos. Ele utiliza uma Fila de Prioridade (*Min-Heap*) para explorar sempre o nó mais próximo que ainda não foi totalmente processado, garantindo uma abordagem gulosa e eficiente. Sua complexidade é: $\mathcal{O}(E \log_2(V))$

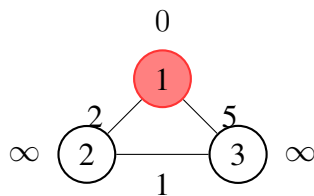
Pseudo-Código:

Algoritmo 3: Algoritmo de Dijkstra

```
1 Função Dijkstra (inicio, adj) :  
2   dist  $\leftarrow$  array de distâncias inicializado com  $\infty$ ;  
3   pq  $\leftarrow$  nova FilaDePrioridade();  
4   dist[inicio]  $\leftarrow$  0;  
5   pq.push((0, inicio));  
6   while enquanto pq não vazia do  
7     (d, u)  $\leftarrow$  pq.top(); pq.pop();  
8     se d > dist[u] então  
9       | continua para a próxima iteração;  
10    fim  
11    para cada (peso, v)  $\in$  adj[u] faça  
12      | se dist[u] + peso < dist[v] então  
13        |   dist[v]  $\leftarrow$  dist[u] + peso;  
14        |   pq.push((dist[v], v));  
15      | fim  
16    fim  
17  end
```

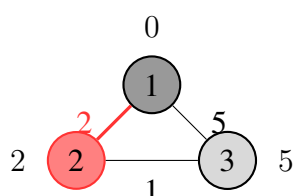
Veja o Dijkstra funcionando, encontrando o caminho mínimo a partir do nó 1:

Passo 1: Início



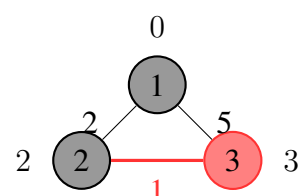
Distância de 1 é 0. Outros são ∞ . Fila: [(0, 1)].

Passo 2: Expandindo 1



Tira 1. Relaxa arestas. *dist*[2] = 2, *dist*[3] = 5. Fila: [(2, 2), (5, 3)].

Passo 3: Expandindo 2



Tira 2. $2 + 1 < 5$. Atualiza *dist*[3] = 3. Fila: [(3, 3), (5, 3)].

O caminho mais curto de 1 para 3 passa por 2, custando 3.

Implementação:

Vejamos a implementação do Dijkstra em Python e C++. Note o uso de *heaps* para otimização.

Python:

```
1 import heapq
2
3 def dijkstra(inicio, adj, n):
4     dist = [float('inf')] * n
5     dist[inicio] = 0
6     pq = [(0, inicio)]
7
8     while pq:
9         d, u = heapq.heappop(pq)
10        if d > dist[u]:
11            continue
12
13        for peso, v in adj[u]:
14            if dist[u] + peso < dist[v]:
15                dist[v] = dist[u] + peso
16                heapq.heappush(pq, (dist[v], v))
17
18    return dist
```

C++:

```
1 vector<int> dijkstra(int inicio, int n, const vector<vector<pair<int, int>>>&
2   adj) {
3     vector<int> dist(n, 1e9);
4     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int,
5     int>>> pq;
6
7     dist[inicio] = 0;
8     pq.push({0, inicio});
9
10    while (!pq.empty()) {
11        int d = pq.top().first;
12        int u = pq.top().second;
13        pq.pop();
14
15        if (d > dist[u]) continue;
16
17        for (auto edge : adj[u]) {
18            int peso = edge.first;
19            int v = edge.second;
20
21            if (dist[u] + peso < dist[v]) {
22                dist[v] = dist[u] + peso;
23                pq.push({dist[v], v});
24            }
25        }
26    }
27    return dist;
28 }
```


3.2.2. Bellman-Ford (Caminho Mínimo com Pesos Negativos)

O algoritmo de Bellman-Ford é utilizado para encontrar caminhos mínimos em grafos que contêm arestas com pesos negativos. Ao contrário do Dijkstra, ele relaxa todas as arestas do grafo repetidas vezes ($V - 1$ vezes, onde V é o número de vértices). Se no V -ésimo passo ainda for possível relaxar alguma aresta, significa que existe um **ciclo de peso negativo** no grafo. Sua complexidade é: $\mathcal{O}(V * E)$

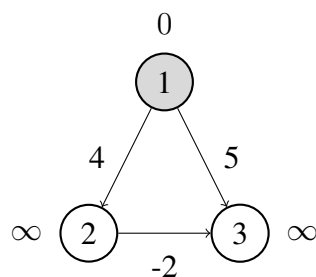
Pseudo-Código:

Algoritmo 4: Algoritmo de BellmanFord

```
1 Função BellmanFord (inicio, arestas,  $V$ ) :  
2    $dist \leftarrow$  array de tamanho  $V$  inicializado com  $\infty$ ;  
3    $dist[inicio] \leftarrow 0$ ;  
4   para  $i \leftarrow 1$  até  $V - 1$  faça  
5     para cada  $(u, v, peso) \in arestas$  faça  
6       se  $dist[u] + peso < dist[v]$  então  
7          $dist[v] \leftarrow dist[u] + peso$ ;  
8       fim  
9     fim  
10  fim  
11  para cada  $(u, v, peso) \in arestas$  faça  
12    se  $dist[u] + peso < dist[v]$  então  
13      retorna Ciclo negativo detectado;  
14    fim  
15  fim
```

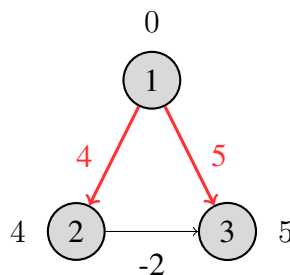
Veja uma iteração de relaxamento do Bellman-Ford:

Passo 1: Estado Inicial



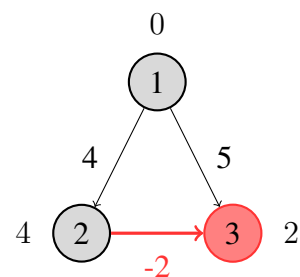
Distâncias iniciais: $dist[1] = 0$, outros ∞ .

Passo 2: Relaxando $1 \rightarrow 2$ e $1 \rightarrow 3$



$dist[2] = 4$, $dist[3] = 5$.

Passo 3: Relaxando $2 \rightarrow 3$



$dist[2] + (-2) < dist[3]$. Logo $dist[3] = 2$.

Implementação:

Python:

```
1 def bellman_ford(inicio, n, arestas):  
2     dist = [float('inf')] * n  
3     dist[inicio] = 0  
4
```

```

5     for _ in range(n - 1):
6         for u, v, peso in arestas:
7             if dist[u] != float('inf') and dist[u] + peso < dist[v]:
8                 dist[v] = dist[u] + peso
9
10        # Verificacao de ciclo negativo
11        for u, v, peso in arestas:
12            if dist[u] != float('inf') and dist[u] + peso < dist[v]:
13                return None
14
15    return dist

```

C++:

```

1 struct Aresta {
2     int u, v, peso;
3 };
4
5 vector<int> bellman_ford(int inicio, int n, const vector<Aresta>& arestas) {
6     vector<int> dist(n, 1e9);
7     dist[inicio] = 0;
8
9     for (int i = 0; i < n - 1; ++i) {
10        for (const auto& a : arestas) {
11            if (dist[a.u] < 1e9 && dist[a.u] + a.peso < dist[a.v]) {
12                dist[a.v] = dist[a.u] + a.peso;
13            }
14        }
15    }
16
17    for (const auto& a : arestas) {
18        if (dist[a.u] < 1e9 && dist[a.u] + a.peso < dist[a.v]) {
19            return {}; // Retorna vetor vazio se houver ciclo negativo
20        }
21    }
22    return dist;
23 }

```

3.2.3. Floyd-Warshall (Todos os Pares)

O algoritmo de Floyd-Warshall resolve o problema do caminho mínimo entre **todos os pares de vértices** do grafo simultaneamente. Ele utiliza Programação Dinâmica, avaliando se o caminho de i para j passando por um vértice intermediário k é menor que o caminho direto conhecido. A complexidade de tempo é $\mathcal{O}(V^3)$.

Pseudo-Código:

Algoritmo 5: Algoritmo de Floyd-Warshall

```
1 Função FloydWarshall (matrizAdj,  $V$ ) :  
2    $dist \leftarrow matrizAdj$ ;  
3   para  $k \leftarrow 0$  até  $V - 1$  faça  
4     para  $i \leftarrow 0$  até  $V - 1$  faça  
5       para  $j \leftarrow 0$  até  $V - 1$  faça  
6         se  $dist[i][k] + dist[k][j] < dist[i][j]$  então  
7            $dist[i][j] \leftarrow dist[i][k] + dist[k][j]$ ;  
8         fim  
9       fim  
10    fim  
11  fim
```

Implementação:

Python:

```
1 def floyd_warshall(n, adj_matrix):  
2     # Cópia a matriz de adjacência  
3     dist = [row[:] for row in adj_matrix]  
4  
5     for k in range(n):  
6         for i in range(n):  
7             for j in range(n):  
8                 if dist[i][k] + dist[k][j] < dist[i][j]:  
9                     dist[i][j] = dist[i][k] + dist[k][j]  
10  
11     return dist
```

C++:

```
1 void floyd_warshall(int n, vector<vector<int>>& dist) {  
2     // dist ja eh inicializada com a matriz de adjacência  
3     for (int k = 0; k < n; ++k) {  
4         for (int i = 0; i < n; ++i) {  
5             for (int j = 0; j < n; ++j) {  
6                 if (dist[i][k] < 1e9 && dist[k][j] < 1e9) {  
7                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);  
8                 }  
9             }  
10        }  
11    }  
12 }
```

3.3. Algoritmos de Árvores Geradoras Mínimas (MST)

Uma Árvore Geradora Mínima (MST) de um grafo conexo, não direcionado e ponderado é um subgrafo que é uma árvore (sem ciclos) e conecta todos os vértices, com o menor peso total de arestas possível ($V - 1$ arestas). Veremos a seguir os dois algoritmos clássicos para resolvê-la.

3.3.1. DSU - Disjoint Set Union (Union-Find)

A estrutura de dados Disjoint Set Union (DSU) é utilizada para rastrear um conjunto de elementos particionados em vários subconjuntos disjuntos (não sobrepostos). Ela suporta duas operações principais quase em tempo constante, $\mathcal{O}(\alpha(V))$, graças às otimizações de **compressão de caminho** e **união por tamanho**: a operação *Find* (descobre a qual conjunto um elemento pertence) e a *Union* (une dois conjuntos).

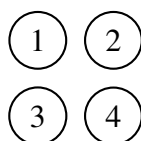
Pseudo-Código:

Algoritmo 6: Disjoint Set Union (DSU)

```
1 Função Find( $x$ ) :  
2   se  $pai[x] == x$  então  
3     retorna  $x$ ;  
4   fim  
5    $pai[x] \leftarrow \text{Find}(pai[x])$  // Compressão de caminho;  
6   retorna  $pai[x]$ ;  
  
7 Função Union( $x, y$ ) :  
8    $raizX \leftarrow \text{Find}(x)$ ;  
9    $raizY \leftarrow \text{Find}(y)$ ;  
10  se  $raizX \neq raizY$  então  
11    // União por tamanho (liga o menor ao maior);  
12    se  $tamanho[raizX] < tamanho[raizY]$  então  
13      Trocar os valores de  $raizX$  e  $raizY$ ;  
14    fim  
15     $pai[raizY] \leftarrow raizX$ ;  
16     $tamanho[raizX] \leftarrow tamanho[raizX] + tamanho[raizY]$ ;  
17  fim
```

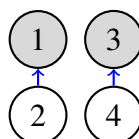
Veja a estrutura do DSU sendo formada (as setas apontam para o pai):

Passo 1: Inicialização



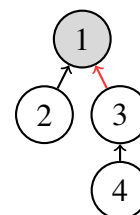
Cada elemento é seu próprio pai.
Nenhum conjunto foi unido ainda.

Passo 2: Union(1, 2) e Union(3, 4)



1 e 3 tornam-se raízes dos seus respectivos conjuntos.

Passo 3: Union(1, 3)



Ambos tinham tamanho 2. O nó 1 se torna o pai do nó 3. Todos agora no mesmo conjunto.

Implementação: Python:

```
1 class DSU:
2     def __init__(self, n):
3         self.pai = list(range(n))
4         self.tamanho = [1] * n
5
6     def find(self, x):
7         if self.pai[x] == x:
8             return x
9         # Compressao de caminho
10        self.pai[x] = self.find(self.pai[x])
11        return self.pai[x]
12
13    def union(self, x, y):
14        raiz_x = self.find(x)
15        raiz_y = self.find(y)
16
17        if raiz_x != raiz_y:
18            # Uniao por tamanho
19            if self.tamanho[raiz_x] < self.tamanho[raiz_y]:
20                raiz_x, raiz_y = raiz_y, raiz_x
21
22            self.pai[raiz_y] = raiz_x
23            self.tamanho[raiz_x] += self.tamanho[raiz_y]
24            return True # Ocorreu uniao
25        return False # Ja estavam no mesmo conjunto
```

C++:

```
1 struct DSU {
2     vector<int> pai, tamanho;
3
4     DSU(int n) {
5         pai.resize(n);
6         tamanho.assign(n, 1);
7         iota(pai.begin(), pai.end(), 0); // Preenche 0, 1, ..., n-1
8     }
9
10    int find(int x) {
11        if (pai[x] == x) return x;
12        return pai[x] = find(pai[x]); // Compressao de caminho
13    }
14
15    bool unite(int x, int y) {
16        int raizX = find(x);
17        int raizY = find(y);
18
19        if (raizX != raizY) {
20            // Uniao por tamanho
21            if (tamanho[raizX] < tamanho[raizY]) swap(raizX, raizY);
22            pai[raizY] = raizX;
23            tamanho[raizX] += tamanho[raizY];
24            return true;
25        }
26        return false;
27    }
28 };
```

3.3.2. Kruskal (Baseado em DSU e Ordenação de Arestas)

O algoritmo de Kruskal ordena todas as arestas do grafo em ordem crescente de peso. Em seguida, itera por cada aresta utilizando uma estrutura de DSU: se os vértices da aresta pertencem a conjuntos diferentes, nós os unimos e adicionamos a aresta à MST. Caso contrário, a aresta é descartada para evitar ciclos. A complexidade de tempo é $\mathcal{O}(E \log E)$ devido à ordenação das arestas.

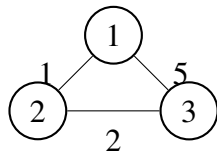
Pseudo-Código:

Algoritmo 7: Algoritmo de Kruskal

```
1 Função Kruskal (arestas, V) :  
2   mst  $\leftarrow \emptyset$ ;  
3   Ordenar arestas em ordem crescente de peso;  
4   Inicializar dsu com V elementos;  
5   para cada (peso, u, v)  $\in$  arestas faça  
6     se dsu.find(u)  $\neq$  dsu.find(v) então  
7       dsu.unite(u, v);  
8       mst.add((u, v, peso));  
9     fim  
10  fim  
11  retorna mst;
```

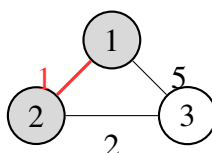
Veja o Kruskal funcionando, selecionando arestas em ordem crescente de peso sem formar ciclos:

Passo 1: Arestas Ordenadas



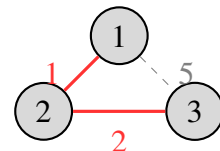
Arestas ordenadas por peso: (1-2, peso 1), (2-3, peso 2), (1-3, peso 5).

Passo 2: Adiciona aresta (1-2)



Menor peso (1). Unimos 1 e 2 via DSU. Aresta aceita na MST.

Passo 3: Adiciona (2-3)



Próxima menor (2). Unimos 2 e 3. Aresta (1-3) é ignorada (formaria ciclo). MST pronta! Custo: 3.

Implementação:

Python:

```
1 def kruskal(n, arestas):  
2     # arestas = lista de tuplas (peso, u, v)  
3     arestas.sort()  
4     dsu = DSU(n)  
5     mst = []  
6     custo_total = 0  
7  
8     for peso, u, v in arestas:  
9         if dsu.unite(u, v):  
10             mst.append((u, v, peso))  
11             custo_total += peso
```

```
12
13     return custo_total, mst
```

C++:

```
1 struct Aresta {
2     int u, v, peso;
3     bool operator<(const Aresta& outro) const {
4         return peso < outro.peso;
5     }
6 };
7
8 int kruskal(int n, vector<Aresta>& arestas) {
9     sort(arestas.begin(), arestas.end());
10    DSU dsu(n);
11    int custoTotal = 0;
12
13    for (auto& a : arestas) {
14        if (dsu.unite(a.u, a.v)) {
15            custoTotal += a.peso;
16        }
17    }
18    return custoTotal;
19 }
```

3.3.3. Prim (Baseado em Fila de Prioridade)

O algoritmo de Prim constrói a MST incrementalmente a partir de um vértice inicial arbitrário. A cada passo, ele escolhe a aresta de menor peso que conecta um vértice que já está na MST a um vértice que ainda não está, utilizando uma Fila de Prioridade (*Min-Heap*). Sua complexidade de tempo é $\mathcal{O}(E \log V)$.

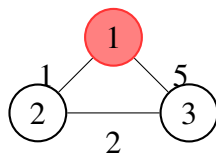
Pseudo-Código:

Algoritmo 8: Algoritmo de Prim

```
1 Função Prim(adj, V) :  
2   visitado  $\leftarrow$  vetor de booleanos falso de tamanho V;  
3   pq  $\leftarrow$  nova FilaDePrioridade() guardando (peso, u);  
4   pq.push((0, 0)) // Começa do vértice 0;  
5   custoTotal  $\leftarrow$  0;  
6   while enquanto pq não vazia do  
7     (peso, u)  $\leftarrow$  pq.top(); pq.pop();  
8     se visitado[u] então  
9       | continuar;  
10    fim  
11    visitado[u]  $\leftarrow$  true;  
12    custoTotal  $\leftarrow$  custoTotal + peso;  
13    para cada (pesoVizinho, v)  $\in$  adj[u] faça  
14      | se  $\neg$ visitado[v] então  
15        | pq.push((pesoVizinho, v));  
16      | fim  
17    fim  
18  end  
19  retorna custoTotal;
```

Veja o Prim crescendo a árvore a partir do vértice 1:

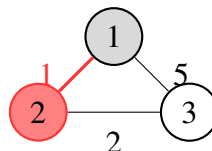
Passo 1: Início no Nó 1



Inicia no vértice 1. Fila de prioridade guarda arestas incidentes: [(1, 2), (5, 3)].

Passo 2: Expande para o

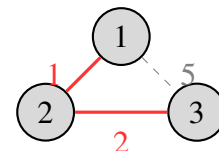
Nó 2



Remove menor aresta (peso 1). Visita o nó 2. Insere vizinhos de 2 na fila: [(2, 3), (5, 3)].

Passo 3: Expande para o

Nó 3



Remove menor aresta (peso 2). Visita o nó 3. Todos conectados! Custo total: 3.

Implementação:

Python:

```
1 import heapq  
2  
3 def prim(n, adj):
```



```

4      # adj[u] = lista de tuplas (peso, v)
5      visitado = [False] * n
6      pq = [(0, 0)] # (peso, vertice)
7      custo_total = 0
8
9      while pq:
10         peso, u = heapq.heappop(pq)
11
12         if visitado[u]:
13             continue
14
15         visitado[u] = True
16         custo_total += peso
17
18         for peso_vizinho, v in adj[u]:
19             if not visitado[v]:
20                 heapq.heappush(pq, (peso_vizinho, v))
21
22     return custo_total

```

C++:

```

1  int prim(int n, const vector<vector<pair<int, int>>>& adj) {
2      vector<bool> visitado(n, false);
3      priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int,
4      int>>> pq;
5
6      pq.push({0, 0}); // {peso, vertice}
7      int custoTotal = 0;
8
9      while (!pq.empty()) {
10         int peso = pq.top().first;
11         int u = pq.top().second;
12         pq.pop();
13
14         if (visitado[u]) continue;
15         visitado[u] = true;
16         custoTotal += peso;
17
18         for (auto& edge : adj[u]) {
19             int pesoVizinho = edge.first;
20             int v = edge.second;
21
22             if (!visitado[v]) {
23                 pq.push({pesoVizinho, v});
24             }
25         }
26     }
27     return custoTotal;
28 }

```

3.4. Algoritmos de Componentes Fortemente Conexas (SCCs) e Pontes de Articulação

Um Componente Fortemente Conexo (SCC - *Strongly Connected Component*) de um grafo direcionado é um subgrafo onde cada par de vértices é alcançável a partir do outro.

3.4.1. Kosaraju (SCC com Duas DFS)

O algoritmo de Kosaraju encontra todos os SCCs de um grafo direcionado realizando duas DFS: a primeira no grafo original para ordenar os vértices por tempo de término (usando uma pilha), e a segunda no grafo transposto (com as arestas invertidas) na ordem da pilha. Sua complexidade de tempo é $\mathcal{O}(V + E)$.

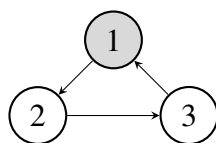
Pseudo-Código:

Algoritmo 9: Algoritmo de Kosaraju

```
1 Função Kosaraju(adj, V) :  
2   visitado  $\leftarrow$  vetor falso de tamanho V;  
3   pilha  $\leftarrow \emptyset$ ;  
4   for i  $\leftarrow 0$  até V - 1 do  
5     se  $\neg$ visitado[i] então  
6       DFS1(i, adj, visitado, pilha);  
7     fim  
8   end  
9   adjT  $\leftarrow$  inverter todas as arestas de adj;  
10  visitado  $\leftarrow$  limpar visitados;  
11  sccs  $\leftarrow \emptyset$ ;  
12  while pilha não vazia do  
13    u  $\leftarrow$  pilha.pop();  
14    se  $\neg$ visitado[u] então  
15      componente  $\leftarrow \emptyset$ ;  
16      DFS2(u, adjT, visitado, componente);  
17      sccs.add(componente);  
18    fim  
19  end  
20  retorna sccs;
```

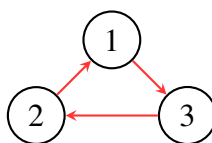
Veja o Kosaraju funcionando em duas etapas:

Passo 1: 1ª DFS (Original)



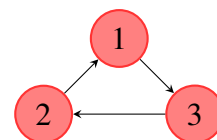
Executa DFS no grafo original e armazena os nós na pilha por tempo de término.

Passo 2: Grafo Transposto



Inverte todas as arestas do grafo original para formar o grafo transposto (*adjT*).

Passo 3: 2ª DFS (SCCs)



Executa DFS baseada na pilha. Cada árvore alcançada forma um SCC isolado.

Implementação:

Python:

```
1 def kosaraju(n, adj):
2     def dfs1(u):
3         visitado[u] = True
4         for v in adj[u]:
5             if not visitado[v]:
6                 dfs1(v)
7         pilha.append(u)
8
9     def dfs2(u, comp):
10        visitado[u] = True
11        comp.append(u)
12        for v in adjT[u]:
13            if not visitado[v]:
14                dfs2(v, comp)
15
16    visitado = [False] * n
17    pilha = []
18    for i in range(n):
19        if not visitado[i]:
20            dfs1(i)
21
22    adjT = [[] for _ in range(n)]
23    for u in range(n):
24        for v in adj[u]:
25            adjT[v].append(u)
26
27    visitado = [False] * n
28    sccs = []
29    while pilha:
30        u = pilha.pop()
31        if not visitado[u]:
32            comp = []
33            dfs2(u, comp)
34            sccs.append(comp)
35    return sccs
```

C++:

```
1 void dfs1(int u, const vector<vector<int>>& adj, vector<bool>& vis, stack<int>&
   st) {
2     vis[u] = true;
3     for (int v : adj[u]) if (!vis[v]) dfs1(v, adj, vis, st);
4     st.push(u);
5 }
6
7 void dfs2(int u, const vector<vector<int>>& adjT, vector<bool>& vis, vector<int>
   & comp) {
8     vis[u] = true;
9     comp.push_back(u);
10    for (int v : adjT[u]) if (!vis[v]) dfs2(v, adjT, vis, comp);
11 }
12
13 vector<vector<int>> kosaraju(int n, const vector<vector<int>>& adj) {
14     vector<bool> vis(n, false);
15     stack<int> st;
16     for (int i = 0; i < n; ++i) if (!vis[i]) dfs1(i, adj, vis, st);
17 }
```

```
18 vector<vector<int>> adjT(n);
19 for (int u = 0; u < n; ++u)
20     for (int v : adj[u]) adjT[v].push_back(u);
21
22 fill(vis.begin(), vis.end(), false);
23 vector<vector<int>> sccs;
24 while (!st.empty()) {
25     int u = st.top(); st.pop();
26     if (!vis[u]) {
27         vector<int> comp;
28         dfs2(u, adjT, vis, comp);
29         sccs.push_back(comp);
30     }
31 }
32 return sccs;
33 }
```

3.4.2. Tarjan (SCC em Uma Única Passada)

O algoritmo de Tarjan encontra os SCCs utilizando apenas uma única DFS. Ele mantém o tempo de descoberta de cada vértice (*tin*) e o menor tempo alcançável (*low*). Utiliza uma pilha auxiliar para agrupar os nós pertencentes ao mesmo componente conexo. Complexidade de tempo: $\mathcal{O}(V + E)$.

Pseudo-Código:

Algoritmo 10: Algoritmo de Tarjan para SCC

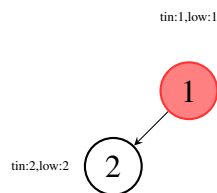
```

1 Função TarjanDFS (u, tin, low, pilha, naPilha, adj) :
2   tin[u]  $\leftarrow$  low[u]  $\leftarrow$  timer;
3   timer  $\leftarrow$  timer + 1;
4   pilha.push(u);
5   naPilha[u]  $\leftarrow$  true;
6   for v  $\in$  adj[u] do
7     se tin[v] == 0 então
8       TarjanDFS (v, tin, low, pilha, naPilha, adj) ;
9       low[u]  $\leftarrow$  min(low[u], low[v]);
10    fim
11    se naPilha[v] então
12      low[u]  $\leftarrow$  min(low[u], tin[v]);
13    fim
14  end
15  se low[u] == tin[u] então
16    // Encontrou a raiz de um SCC;
17    while topo  $\neq$  u do
18      topo  $\leftarrow$  pilha.pop();
19      naPilha[topo]  $\leftarrow$  false;
20      adiciona topo ao SCC atual;
21    end
22  fim

```

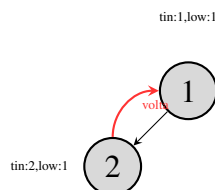
Veja o Tarjan utilizando tempos de descoberta e valores de *low*:

Passo 1: Descoberta
(*tin*, *low*)



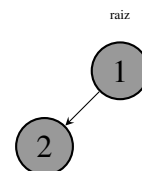
Visita o nó 1, define $tin[1] = low[1] = 1$ e empilha. Vai para o nó 2.

Passo 2: Back-edge e Low



Aresta de retorno de 2 para 1 atualiza $low[2] = \min(low[2], tin[1]) = 1$.

Passo 3: Raiz do SCC



Como $low[1] == tin[1]$, 1 é raiz de um SCC. Desempilha os nós até 1.

Implementação:

C++:

```
1 int timer = 0;
2 void tarjanDFS(int u, const vector<vector<int>>& adj, vector<int>& tin, vector<
   int>& low, stack<int>& st, vector<bool>& inStack, vector<vector<int>>& sccs
   ) {
3     tin[u] = low[u] = ++timer;
4     st.push(u);
5     inStack[u] = true;
6
7     for (int v : adj[u]) {
8         if (tin[v] == -1) {
9             tarjanDFS(v, adj, tin, low, st, inStack, sccs);
10            low[u] = min(low[u], low[v]);
11        } else if (inStack[v]) {
12            low[u] = min(low[u], tin[v]);
13        }
14    }
15
16    if (low[u] == tin[u]) {
17        vector<int> comp;
18        while (true) {
19            int v = st.top(); st.pop();
20            inStack[v] = false;
21            comp.push_back(v);
22            if (u == v) break;
23        }
24        sccs.push_back(comp);
25    }
26 }
```

3.5. Algoritmos de Fluxo Máximo e Emparelhamento Bipartido

Problemas de fluxo máximo modelam redes de transporte onde cada aresta possui uma capacidade máxima. O objetivo é enviar o máximo de "fluxo" possível de uma fonte (S) até um sumidouro (T).

3.5.1. Dinic (Fluxo Máximo Eficiente)

O algoritmo de Dinic utiliza o conceito de *grafo de níveis* (construído via BFS) para encontrar caminhos de aumento e *fluxos bloqueantes* (encontrados via DFS) em camadas. Sua complexidade de tempo é $\mathcal{O}(V^2E)$, sendo extremamente rápido em grafos gerais.

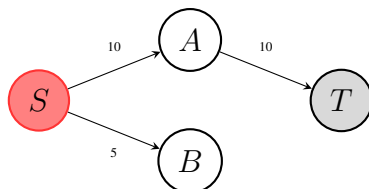
Pseudo-Código:

Algoritmo 11: Algoritmo de Dinic

```
1 Função Dinic( $s, t, adj$ ) :  
2    $fluxoMaximo \leftarrow 0$ ;  
3   while BFS constrói o grafo de níveis a partir de  $s$  até  $t$  do  
4     while encontrar caminho de fluxo bloqueante com DFS do  
5        $fluxoMaximo \leftarrow fluxoMaximo + fluxoEncontrado$ ;  
6     end  
7   end  
8   retorna  $fluxoMaximo$ ;
```

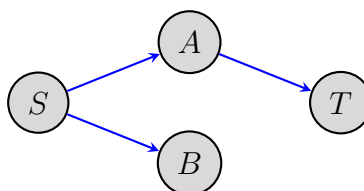
Veja o Dinic construindo o grafo de níveis e encontrando o fluxo bloqueante:

Passo 1: Rede Residual



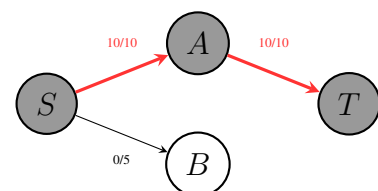
Rede inicial com capacidades nas arestas entre a fonte S e o sumidouro T .

Passo 2: Grafo de Níveis (BFS)



BFS atribui níveis aos nós ($level[S] = 0, level[A] = 1, level[T] = 2$).

Passo 3: Fluxo Bloqueante (DFS)



DFS envia fluxo pelas camadas até que nenhuma rota adicional seja possível.

Implementação:

C++ (Estrutura Completa de Dinic):

```
1 struct Edge {  
2     int to;  
3     long long cap;  
4     long long flow;  
5     int rev;  
6 };  
7  
8 struct Dinic {  
9     int n;  
10    vector<vector<Edge>> adj;  
11    vector<int> level, ptr;
```

```

12
13 Dinic(int n) : n(n), adj(n), level(n), ptr(n) {}
14
15 void add_edge(int from, int to, long long cap) {
16     adj[from].push_back({to, cap, 0, (int)adj[to].size()});
17     adj[to].push_back({from, 0, 0, (int)adj[from].size() - 1});
18 }
19
20 bool bfs(int s, int t) {
21     fill(level.begin(), level.end(), -1);
22     level[s] = 0;
23     queue<int> q;
24     q.push(s);
25     while (!q.empty()) {
26         int v = q.front();
27         q.pop();
28         for (auto& edge : adj[v]) {
29             if (edge.cap - edge.flow > 0 && level[edge.to] == -1) {
30                 level[edge.to] = level[v] + 1;
31                 q.push(edge.to);
32             }
33         }
34     }
35     return level[t] != -1;
36 }
37
38 long long dfs(int v, int t, long long pushed) {
39     if (pushed == 0) return 0;
40     if (v == t) return pushed;
41     for (int& cid = ptr[v]; cid < adj[v].size(); ++cid) {
42         auto& edge = adj[v][cid];
43         int tr = edge.to;
44         if (level[v] + 1 != level[tr] || edge.cap - edge.flow == 0)
45             continue;
46         long long tr_pushed = dfs(tr, t, min(pushed, edge.cap - edge.flow));
47         if (tr_pushed == 0) continue;
48         edge.flow += tr_pushed;
49         adj[tr][edge.rev].flow -= tr_pushed;
50         return tr_pushed;
51     }
52     return 0;
53 }
54
55 long long max_flow(int s, int t) {
56     long long flow = 0;
57     while (bfs(s, t)) {
58         fill(ptr.begin(), ptr.end(), 0);
59         while (long long pushed = dfs(s, t, 1e18)) {
60             flow += pushed;
61         }
62     }
63     return flow;
64 };

```


3.5.2. Algoritmo de Kuhn (Emparelhamento Bipartido Máximo)

O emparelhamento bipartido máximo (*Maximum Bipartite Matching*) encontra o maior conjunto possível de arestas em um grafo bipartido onde nenhuma aresta compartilha um vértice final. O Algoritmo de Kuhn resolve isso utilizando caminhos de aumento via DFS de forma muito elegante. Complexidade: $\mathcal{O}(V \cdot E)$.

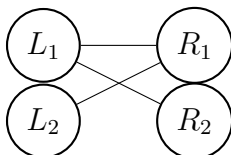
Pseudo-Código:

Algoritmo 12: Algoritmo de Kuhn para Emparelhamento Bipartido

```
1 Função try_kuhn ( $v, adj, visitado, emparelhamento$ ) :  
2   if visitado[ $v$ ] then  
3     retorna falso;  
4   end  
5   visitado[ $v$ ]  $\leftarrow$  true;  
6   para cada  $to \in adj[v]$  faça  
7     se emparelhamento[ $to$ ] == -1 ou  
       try_kuhn (emparelhamento[ $to$ ],  $adj, visitado, emparelhamento$ )  
       então  
8       emparelhamento[ $to$ ]  $\leftarrow v$ ;  
9       retorna verdadeiro;  
10  fim  
11 fim  
12 retorna falso;
```

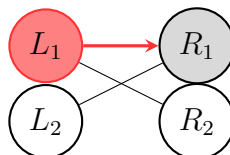
Veja o Algoritmo de Kuhn encontrando o emparelhamento máximo:

Passo 1: Lados L e R



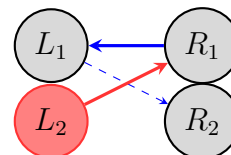
Grafo dividido em dois conjuntos disjuntos L e R . Nenhum casamento inicial.

Passo 2: Tentativa de Casamento



L_1 tenta casar com R_1 . Como R_1 está livre, o casamento é feito com sucesso.

Passo 3: Caminho de Aumento



L_2 quer R_1 (ocupado). L_1 muda para R_2 , liberando R_1 para L_2 !

Implementação:

C++:

```
1 vector<vector<int>>> adj;
2 vector<int> mt;
3 vector<bool> visited;
4
5 bool try_kuhn(int v) {
6     if (visited[v]) return false;
7     visited[v] = true;
8     for (int to : adj[v]) {
9         if (mt[to] == -1 || try_kuhn(mt[to])) {
10             mt[to] = v;
11             return true;
12         }
13     }
14     return false;
15 }
16
17 int max_bipartite_matching(int n, int m) {
18     //n = nos do lado esquerdo, m = nos do lado direito
19     mt.assign(m, -1);
20     int count = 0;
21     for (int v = 0; v < n; ++v) {
22         visited.assign(n, false);
23         if (try_kuhn(v)) ++count;
24     }
25     return count;
26 }
```

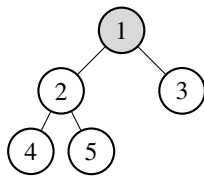
3.6. Algoritmos mais Complexos

3.6.1. LCA - Lowest Common Ancestor (Menor Ancestral Comum)

O LCA de dois nós u e v em uma árvore enraizada é o nó mais profundo que é ancestral de ambos. O método de *Binary Lifting* (Içamento Binário) pré-calcula os ancestrais de potências de 2 em $\mathcal{O}(N \log N)$, permitindo responder a cada consulta de LCA em $\mathcal{O}(\log N)$.

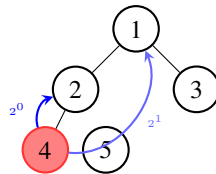
Veja o LCA por Içamento Binário em ação:

Passo 1: Árvore Enraizada



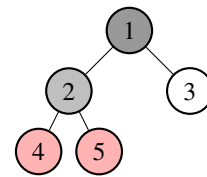
Árvore com 5 nós. O nó 1 é definido como a raiz.

Passo 2: Pré-cálculo (up)



Armazena ancestrais de potências de 2 ($up[4][0] = 2, up[4][1] = 1$).

Passo 3: Consulta de LCA



O LCA entre os nós 4 e 5 é o nó 2 (ancestral comum mais profundo).

Implementação:

C++:

```
1 int n, l;
2 vector<vector<int>>> adj;
3 vector<vector<int>>> up;
4 vector<int> tin, tout;
5 int timer;
6
7 void dfs(int v, int p) {
8     tin[v] = ++timer;
9     up[v][0] = p;
10    for (int i = 1; i <= l; ++i)
11        up[v][i] = up[up[v][i - 1]][i - 1];
12    for (int u : adj[v]) {
13        if (u != p) dfs(u, v);
14    }
15    tout[v] = ++timer;
16 }
17
18 bool is_ancestor(int u, int v) {
19     return tin[u] <= tin[v] && tout[u] >= tout[v];
20 }
21
22 int lca(int u, int v) {
23     if (is_ancestor(u, v)) return u;
24     if (is_ancestor(v, u)) return v;
25     for (int i = l; i >= 0; --i) {
26         if (!is_ancestor(up[u][i], v))
27             u = up[u][i];
28     }
29     return up[u][0];
30 }
31
32 void preprocess(int root) {
```

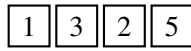
```
33     tin.resize(n); tout.resize(n);
34     timer = 0;
35     l = ceil(log2(n));
36     up.assign(n, vector<int>(l + 1));
37     dfs(root, root);
38 }
```

3.6.2. Segment Tree (Árvore de Segmentos)

A Segment Tree é uma estrutura de dados binária que armazena intervalos ou segmentos. Ela permite realizar consultas de intervalo (como soma, mínimo, máximo) e atualizações de elementos ou intervalos em tempo $\mathcal{O}(\log N)$.

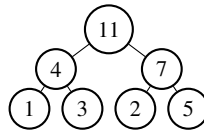
Veja a Segment Tree construída sobre um vetor de 4 elementos:

Passo 1: Vetor Base



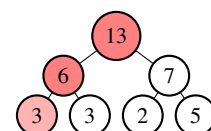
Vetor inicial com 4 elementos ($N = 4$).

Passo 2: Árvore Binária



Cada nó pai armazena a soma dos filhos (Total = 11, total de 7 nós).

Passo 3: Atualização



Atualiza a posição 0 de 1 para 3. Propaga os valores em $\mathcal{O}(\log N)$.

Implementação:

C++:

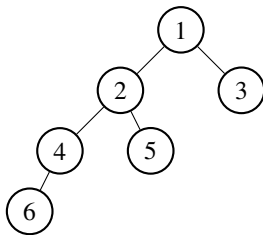
```
1 struct SegTree {
2     int n;
3     vector<long long> tree;
4
5     SegTree(int n) : n(n), tree(4 * n, 0) {}
6
7     void update(int node, int start, int end, int idx, long long val) {
8         if (start == end) {
9             tree[node] = val;
10            return;
11        }
12        int mid = (start + end) / 2;
13        if (idx <= mid) update(2 * node, start, mid, idx, val);
14        else update(2 * node + 1, mid + 1, end, idx, val);
15        tree[node] = tree[2 * node] + tree[2 * node + 1];
16    }
17
18    long long query(int node, int start, int end, int l, int r) {
19        if (r < start || end < l) return 0;
20        if (l <= start && end <= r) return tree[node];
21        int mid = (start + end) / 2;
22        return query(2 * node, start, mid, l, r) + query(2 * node + 1, mid + 1,
23            end, l, r);
24    }
25};
```

3.6.3. HLD - Heavy-Light Decomposition

O HLD decompõe uma árvore em uma coleção de caminhos disjuntos (cadeias). Ele classifica as arestas em *Heavy* (para o filho com a maior subárvore) e *Light*. Isso garante que qualquer caminho entre dois nós na árvore cruza no máximo $\mathcal{O}(\log N)$ arestas *light*. Combinado com uma Segment Tree, permite resolver consultas complexas em caminhos de árvores em $\mathcal{O}(\log^2 N)$.

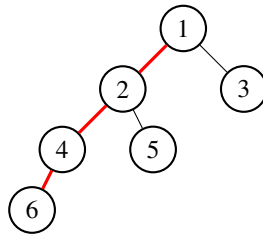
Veja a Decomposição Heavy-Light em uma árvore de 6 nós:

Passo 1: Tamanho Subárvores



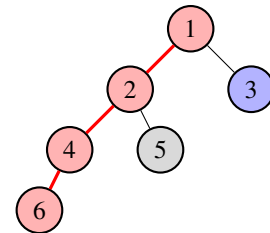
Calcula o tamanho das subárvores de cada filho para determinar a aresta pesada.

Passo 2: Arestas Heavy



Arestas pesadas (*heavy*, em vermelho) vão para o filho com maior subárvore.

Passo 3: Cadeias Disjuntas



Árvore decomposta em cadeias lineares para consultas em $\mathcal{O}(\log^2 N)$.

Implementação:

C++:

```
1 vector<int> parent, depth, heavy, head, pos;
2 int cur_pos;
3
4 int dfs_hld(int v, vector<vector<int>>& adj) {
5     int size = 1, max_c_size = 0;
6     heavy[v] = -1;
7     for (int u : adj[v]) {
8         if (u != parent[v]) {
9             parent[u] = v;
10            depth[u] = depth[v] + 1;
11            int c_size = dfs_hld(u, adj);
12            size += c_size;
13            if (c_size > max_c_size) {
14                max_c_size = c_size;
15                heavy[v] = u;
16            }
17        }
18    }
19    return size;
20 }
21
22 void decompose(int v, int h, vector<vector<int>>& adj) {
23     head[v] = h;
24     pos[v] = cur_pos++;
25     if (heavy[v] != -1) decompose(heavy[v], h, adj);
26     for (int u : adj[v]) {
27         if (u != parent[v] && u != heavy[v]) decompose(u, u, adj);
28     }
29 }
30 }
```

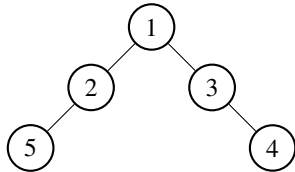
```
31 void init_hld(int n, vector<vector<int>>& adj) {  
32     parent.resize(n); depth.resize(n); heavy.assign(n, -1);  
33     head.resize(n); pos.resize(n);  
34     cur_pos = 0;  
35     dfs_hld(0, adj);  
36     decompose(0, 0, adj);  
37 }
```

3.6.4. Centroid Decomposition (Decomposição em Centróide)

A Decomposição em Centróide divide recursivamente uma árvore encontrando o centróide (o nó cuja remoção deixa todas as subárvores com tamanho $\leq N/2$). Isso reduz a altura da árvore de decomposição para $\mathcal{O}(\log N)$, sendo a ferramenta definitiva para resolver problemas de caminhos e distâncias entre quaisquer pares de nós na árvore.

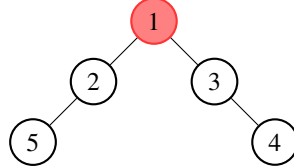
Veja a Decomposição em Centróide reduzindo a árvore:

Passo 1: Árvore Original



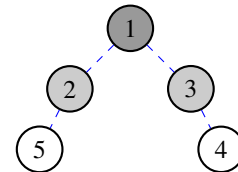
Árvore geral com 5 nós conectados.

Passo 2: Achar Centróide



Identifica o nó 1 como centróide (nenhuma subárvore excede $N/2$).

Passo 3: Árvore de Centróides



Remove o centróide e repete recursivamente, formando a árvore de centróides (altura $\leq \log N$).

Implementação:

C++:

```
1 vector<vector<int>>> adj;
2 vector<bool> removed;
3 vector<int> sub_size;
4
5 void get_sub_size(int u, int p) {
6     sub_size[u] = 1;
7     for (int v : adj[u]) {
8         if (v != p && !removed[v]) {
9             get_sub_size(v, u);
10            sub_size[u] += sub_size[v];
11        }
12    }
13 }
14
15 int get_centroid(int u, int p, int total) {
16     for (int v : adj[u]) {
17         if (v != p && !removed[v] && sub_size[v] > total / 2)
18             return get_centroid(v, u, total);
19     }
20     return u;
21 }
22
23 void decompose(int u) {
24     get_sub_size(u, u);
25     int centroid = get_centroid(u, u, sub_size[u]);
26
27     removed[centroid] = true;
28
29     // Processa o centróide atual (ex: caminhos que passam por ele)
30
31     for (int v : adj[centroid]) {
32         if (!removed[v]) decompose(v);
33     }
```